

RESEARCH ARTICLE

Intent-Based Multi-Cloud Storage Management Powered by a Fine-Tuned Large Language Model

JINGYA ZHENG¹, GAOFENG TAO^{1,2}, SHUXIN QIN², DAN WANG², AND ZHONGJUN MA¹¹Shandong Future Network Research Institute, Jinan 250003, China²Purple Mountain Laboratories, Nanjing 211111, China

Corresponding author: Gaofeng Tao (taogaofeng@pmlabs.com.cn)

This work was supported in part by Taishan Industrial Experts Program under Grant TSCX202306120.

ABSTRACT Storage resources are essential in heterogeneous multi-cloud environments. In response to the growing demand for efficient storage resource management (SRM) in these environments, this paper proposes an intent-based storage management (IBSM) system powered by a fine-tuned large language model (LLM). To overcome the limitations of existing methods, the IBSM system focuses on enhancing the controllability, completeness, and reliability of SRM in multi-cloud environments. Specifically, the IBSM system employs a dual-phase joint intent classification algorithm, which leverages a fine-tuned LLM to accurately identify user intents across diverse knowledge backgrounds. Additionally, the system constructs a collaborative intent decomposition method, which guarantees the integrity of intents. Furthermore, the system integrates an automated intent deployment mechanism that supports error recovery through checkpoints. Experimental results show that the system achieves a whole end-to-end (E2E) lifecycle for managing user intents. The E2E time is reduced by at least half compared to the manual approach, with an average of 50.14% dedicated to interactive tasks. Performance metrics for intent classification, including accuracy, precision, and recall, all exceed 90%. Moreover, the recovery time is reduced by an average of 30.6%. Therefore, the system provides a valuable solution for the autonomous management of multi-cloud resources.

INDEX TERMS Autonomous management, cloud computing, cloud-network integration, intent-based networking, large language model, multi-cloud.

I. INTRODUCTION

China's east-west computing resources transmission (EWCRT) project is a unique and innovative path toward developing China's green digital economy [1]. As a core technology within the EWCRT Project, multi-cloud computing has recently regained significant attention from the global telecommunications industry [2].

Multi-cloud computing employs services from various cloud vendors to execute an application either sequentially or concurrently [3], [4]. Initially, multi-cloud computing emerged as a solution to the challenges associated with reliance on a single vendor. These challenges arise from the wide geographic distribution, high interdependencies, and complex systems [5]. Multi-cloud computing enables the selection of services based on geographic proximity to end

users, which contributes to performance, flexibility, redundancy and cost-effectiveness [6].

A multi-cloud computing environment consolidates heterogeneous resources, such as computing, networking, and storage. Among these, storage resources serve as the fundamental medium for business data and are integral to the infrastructure of data centers. Moreover, they play an essential role in safeguarding the lifeline of the digital economy.

The management of multi-cloud storage resources presents significant challenges, primarily driven by two key factors. On the one hand, the EWCRT project necessitates efficient and cost-effective resource management to tackle its inherent complexities. On the other hand, the exponential surge in data volume has markedly increased the difficulty of managing storage resources across multi-cloud environments. According to a report by the international data corporation (IDC), the volume of data generated by enterprises is expected to grow from 59 zettabytes in 2022 to 155 zettabytes by 2026 [7].

The associate editor coordinating the review of this manuscript and approving it for publication was Hadi Tabatabaee Malazi¹.

Manual management methods are inherently inefficient, time-consuming, and error-prone, because they require manual configuration and continuous maintenance. Furthermore, these methods often necessitate a high level of professional expertise, which poses significant challenges for non-specialist users. Thus, there is an urgent demand for an intent-driven management system that incorporates built-in automation and a proactive approach to enhance end-to-end (E2E) management.

Although limited research has focused on multi-cloud environments, studies on intent-based networking (IBN) in cloud networking offer valuable insights. A comprehensive overview of relevant studies is provided in Section II, while a concise summary is presented below. Initially, IBN frameworks utilized formal languages to articulate intents. Later approaches incorporated natural language processing (NLP) techniques to reduce the level of formality. Common techniques employed in the approaches included advanced methods such as long short-term memory (LSTM) networks, RASA, and reinforcement learning (RL). However, these studies are predominantly vendor-specific and frequently encounter difficulties in accurately identifying high-level user intents. In addition, some studies primarily rely on prompts to guide general-purpose large language models (LLMs), such as the ChatGPT series, which face challenges with controllability [8]. Moreover, the fine-tuning method has been employed for open-source LLMs in the domain of IBN.

Compared to IBN management (IBNM), intent-based storage management (IBSM) introduces distinct challenges. Firstly, translating original intents into intricate storage operations within a multi-cloud environment proves to be complex. Secondly, integrating intelligent error recovery with automated deployment exacerbates the difficulty. The root causes of these issues are analyzed in Section III. To tackle the challenges present in existing methods, we propose an IBSM system for multi-cloud environments. The three main contributions of this work are as follows.

The first is the dual-phase joint intent classification algorithm, which is responsible for identifying the diverse intents of all users, including those with limited domain knowledge. In the first stage, intents are classified into different storage modes using the fine-tuned LLM. In the second stage, intents are further categorized into several basic tasks of the multi-cloud platform based on the identified modes. Consequently, the algorithm notably enhances the efficiency of recognizing the user intents in the multi-cloud storage domain.

The second contribution is a collaborative intent decomposition method that directly translates user intents into imperative policies compatible with multiple cloud vendors. This method constructs an atomic-level policy tree based on classification results. Each node in the tree represents a basic task of multi-cloud platforms. The specified LLM assists users in providing missing parameter values. These values are validated against real-time data from various cloud

providers. This method significantly enhances the completeness of intent decomposition.

The third one is an automated intent deployment scheme that manages the execution order and handles errors. In this scheme, the nodes of a fine-grained policy tree function as the basic units of deployment. The scheme employs closed-loop control finite state machines (FSMs) at all levels to automate the sequential execution of the tree. In the case of a recoverable error, the scheme performs checkpoint recovery using states in the FSMs associated with error handling. This scheme ensures reliable deployment across heterogeneous multi-cloud platforms.

The remainder of this paper is organized as follows. Section II provides an overview of previous studies on similar research avenues. Section III details the methodology used to develop the proposed system. Section IV presents the results. Section V provides a discussion of these results. Finally, Section VI summarizes the main conclusions.

II. RELATED WORK

The IBN aims to simplify complex configurations using an intent-driven approach [9], [10]. An IBNM system enables users to express high-level management objectives in natural language without detailing the specific implementation steps within the underlying infrastructure [11]. IBNM functions can be broadly categorized into intent fulfillment, and intent assurance [12]. Fulfillment involves intent ingestion, translation, and orchestration, focusing on providing the necessary functions and interfaces for the intent to be conveyed to the network and implemented. Complementary to the fulfillment, assurance deals with functions and interfaces allowing users to verify that the network indeed meets the specified targets set by the fulfillment [13], [14], [15], [16]. These two categories are typically studied separately. Our focus is primarily on the first category, and related work will be introduced in this section. Research on the second category, intent assurance, is outside the scope of this paper.

A. TRADITIONAL METHODS

Originally, IBN frameworks relied on formal languages to express intents and provided tools for optimizing the network configurations to satisfy them [17], [18], [19]. Formal languages offer a precise and unambiguous method for specifying network policies and configurations, thereby minimizing the risk of misinterpretation. Nonetheless, their use may fail to capture certain aspects of network behavior, which could result in oversimplification.

Subsequent approaches integrated NLP tools to reduce formality. NLP tools allow network operators to articulate their intents in natural language, making the process more intuitive and accessible. These tools automate the conversion of natural language inputs into network configurations, thereby reducing manual effort and minimizing the potential for errors.

NAIL employed NLP techniques in conjunction with a predefined dictionary incorporating regex and keyword recognition methods to distinguish network elements linked to the intent from the primary objective [20]. 5G-CLARITY provided an NLP interface for private 5G network operators, featuring an intent engine to process intents and an artificial intelligence (AI) engine with ML models [21].

LSTM networks have been applied in IBN. LUMI, an intent-based framework, leveraged NLP to identify entities from operator utterances via a chatbot interface. The framework employed LSTM for entity extraction, and these entities were then used to compose a network intent language (Nile), later compiled with standard network programming languages [22]. Nile is a user-friendly intent language, offering automatic deployment of connectivity intents with middlebox support [23]. In the field of IBN, some studies frequently use it as an intermediary component between natural language and the underlying configuration. For instance, user intents are input through Google assistant, then translated into Nile intents, and finally converted into network configurations [24].

Bidirectional long short-term memory (Bi-LSTM) networks, an advanced extension of LSTM, are extensively employed for IBN. OPINE, a framework utilizing a neural model with Bi-LSTM and conditional random fields (CRFs), extracted user intents from natural language utterances and translated them into a formal intent language via a chat-based interface [25]. SMART proposed an intent-driven network management architecture to translate, verify, and map intents into policies. The intent translation module utilized Bi-LSTM and morphological rules to convert the natural language expression of intents into a network-understandable and regularized format [26].

RASA is a conversational AI framework that natively supports multiple models, including LSTM. EVIAN was an intent-rendering platform that extracted entities through a chatbot interface built with RASA. The platform leveraged LSTM to identify actions and entities within user intents [27]. Some studies utilize RASA without specifying the model used. For instance, Chat-IBN-RASA enabled users to interact with the IBNM system via natural language, using the RASA chatbot without requiring in-depth technical knowledge [28].

RL methods are also prevalent in related research. A Q-learning-based RL approach was used to translate intents into efficient resource allocations within cloud infrastructure [29].

Although these methods exhibit some learning capabilities, they struggle to comprehend complex multi-turn conversations (MCs). This difficulty arises primarily from the complexities of domain-specific vocabulary in cloud networking. As a result, the quality of the responses generated by these techniques remains constrained.

B. METHODS BASED ON LLMs

LLMs have shown remarkable capabilities in generating fluent and relevant natural texts for IBN [30], [31], [32].

These abilities stem from the vast amount of real-world data on which they are trained. Consequently, studies on the use of LLMs in IBNM systems are emerging. LLM methods typically utilize prompt engineering, retrieval-augmented generation (RAG), human feedback (HF) models, and fine-tuning.

1) PROMPT ENGINEERING

In the existing literature, the dominant approach for LLMs involves utilizing prompt engineering techniques with general-purpose ChatGPT series. Prompt engineering refers to the design of input prompts aimed at optimizing the output of language models. Its main advantages include enhanced response accuracy, task relevance, and efficiency, all achieved without the necessity for extensive model retraining.

Early studies guided the GPT-3.5 model to produce desired outputs through strategic prompting [33]. For example, GPT-3.5 was selected for fully automated network intelligence [34]. An implementation of the IBNM framework was explored using few-shot learning on GPT-3.5 [35]. S-Witch converted user inputs related to switch configuration into actual configuration commands by connecting GPT-3.5 through step-by-step prompting [36].

Recent research predominantly utilizes GPT-4, the most advanced model in the ChatGPT series, to assist users with network configurations [37], [38], [39]. NetBuddy leveraged the few-shot learning capabilities of GPT-4 to translate natural language into low-level device configurations using self-healing. The self-healing process incorporated a verifier component to verify the output of each LLM and to provide feedback in case it detects errors, which is an essential component for a fully automated pipeline [40]. NetConfEval presented two functional GPT-4-based prototypes capable of generating fully working low-level configurations [41]. GeNet used GPT-4 and prompt engineering to interpret and update topologies and configurations based on intents [42].

Several studies mention models from the ChatGPT series without specifying their exact versions. EMERGENCE proposed an IBNM system based on OpenAI's ChatGPT, which utilized prompt engineering to classify user intents and generate policy trees. The system also incorporated closed-loop control for intent deployment, establishing a solid foundation for the development of our system prototype [43], [44], [45]. A detailed analysis of this approach is provided in Section V. NetLM introduced a framework that leveraged the ChatGPT series and prompt-based techniques to generate imperative policies. This framework progressively refined user intents through multiple abstraction levels, including declarative, deterministic, and imperative policies. Moreover, policy trees were employed to model the decomposition structure and execution sequence [46]. However, the multi-level approach of NetLM increased both the complexity of the policy trees and the overall system. SAI analyzed user intents using prompted LLMs and broke them down into structured tasks [47].

Some studies have explored LLMs beyond the ChatGPT series. The few-shot learning capabilities of Google Bard was used to generate a management code for an edge cloud infrastructure [48], [49]. Google Bard was also leveraged to create task-specific code for graph analysis and manipulation [50]. Llama2 was used to evaluate the framework on computer networks, incorporating the network graph directly into the prompt through in-context learning [51], [52]. Mistral-7B was introduced to achieve enhanced performance in LLM-assisted IBNM [53], [54]. A three-fold strategy involving intent processing and assurance was proposed, which is associated with the BERT model [55]. NFV-Intent used both GPT-3.5 and six other offline LLMs to convert language intents into JSON configurations [56]. LLM-NetCFG employs a Zephyr model to translate high-level network intents into specific device configurations using prompt engineering [57], [58].

These studies depend on well-designed prompts, primarily using models from the ChatGPT series, to enable functions such as intent recognition, intent translation, and intent deployment, thereby offering valuable insights for the application of LLMs in IBNM. Nevertheless, for certain domain-specific and professional tasks, relying solely on prompt engineering may not be sufficient to guide the model in retrieving relevant information. Therefore, prompt engineering is frequently combined with other LLM techniques.

2) RAG AND HF

In the context of generative AI and language model applications, prompt engineering, RAG, and HF each serve distinct roles and provide unique advantages. They can complement each other to improve model performance. RAG integrates external information retrieval with generative models, enabling the generation of more accurate and contextually relevant responses for domain-specific tasks. By dynamically retrieving information from a knowledge base, RAG addresses the limitations of prompt engineering. HF uses human evaluators to guide the model through RL or direct corrections. Its primary benefit lies in improving the model's alignment with human expectations, ensuring more reliable, user-centered outputs by explicitly incorporating human preferences.

Several studies have explored the combination of prompt engineering and RAG. Prompts and RAG techniques were combined to translate natural language intents into network configurations [59]. The effectiveness of ChatGPT using RAG and prompts was showcased in [60]. The performing prompt engineering and domain knowledge library for GPT-series models was illustrated in [61]. NetAssistant applied RAG and few-shot prompting to provide network diagnosis workflows.

Other studies have examined the integration of all three approaches. For instance, an intent classification system based on Code Llama was introduced to simplify network

configuration and management, employing prompt engineering, RAG, and a HF loop [62], [63].

Although general-purpose LLMs that integrate prompt engineering with RAG and HF have shown strong performance across various NLP tasks, they are not specifically designed for automating resource management. Furthermore, closed-source LLMs have raised concerns related to controllability, accessibility, and security.

3) FINE-TUNING

To tackle the above challenge, several studies have focused on fine-tuning open-source LLMs. Fine-tuning enhances model adaptation to specific tasks by leveraging task-specific datasets. Compared to RAG and HF techniques, fine-tuning offers a more robust approach to improving model accuracy and controllability, particularly in specialized domains. For example, Mobile-Llama was built by instruction fine-tuning the Llama2-13B model. Mobile-Llama offered three main functions: packet analysis, IP routing analysis, and performance analysis, primarily contributing to the automation for 5G network management [64], [65].

Similarly, NetLLM designed an efficient data-driven low-rank networking adaptation (DD-LRNA) scheme to drastically reduce the fine-tuning costs. NetLLM served as a multimodal model for multiple networking tasks, including viewport prediction, adaptive bitrate streaming, and cluster job scheduling [66].

Additionally, NetGPT leveraged a low-rank adaptation (LoRA) technique, a parameter efficient fine-tuning (PEFT) technique, to achieve parameter-efficient fine-tuning on a consumer-level hardware. NetGPT deployed smaller edge LLMs for the edge while larger one for the cloud, and meaningfully realized collaborative cloud-edge computing to provision personalized content generation services [67], [68].

The studies underscore the considerable potential of utilizing fine-tuned LLMs within IBNM. Additionally, they introduce self-constructed datasets for network analysis and resource allocation, which are further discussed in Section III. Despite the notable performance of these fine-tuned models in IBN, their applicability to multi-cloud computing environments remains limited.

In addition, some studies fail to account for the complexity of intents expressed in natural language, neglecting the need to decompose the intents of laymen into a hierarchical structure of actions. Moreover, many studies overlook the partial nature of the original intents. Besides, some focus exclusively on executing API calls, without addressing verification or error recovery.

The IBSM system differs from current state-of-the-art systems by addressing the challenges of a heterogeneous multi-cloud environment. The system spans all stages of intent management. The system fine-tuned an open-source language model with fewer than ten billion parameters by employing datasets dedicated to multi-cloud storage. Through prompt engineering, the fine-tuned LLM can

classify various user intents and assist the system in filling the missing parameter values of a policy tree. Moreover, the system verifies the intent execution and performs checkpoint recovery.

III. METHODOLOGY

A. OVERALL ARCHITECTURE

The IBSM system takes natural language intents as inputs and automatically performs the corresponding management actions. In the system, the lifecycle of an intent begins at the user end and finishes at the end of the platform. The user end integrates a multi-cloud application ecosystem. The platform end functions as an infrastructure that supports multi-cloud services from heterogeneous cloud vendors. A description of our multi-cloud platform is provided in Section IV [69].

The system consists of three subsystems, a user interface (UI), and a multi-cloud storage LLM (MS-LLM). The three subsystems are intent classification, decomposition, and deployment. The intent classification subsystem comprises a dual-phase joint intent classification algorithm. The intent decomposition subsystem is primarily designed to convert incomplete user intents into fine-grained policy trees. The intent deployment subsystem ensures the reliable operation of the policy trees. The training method and dataset for the LLM are elaborated on later in this section.

The UI is developed using the Gradio framework, which operates similarly to a chatbot. This facilitates bidirectional communication with users. On one hand, the interface accepts natural language text inputs, which may include user intents, additional context, feedback, and other conversational elements. These inputs are initially processed by the intent classification subsystem that categorizes them into specific intent groups. If the subsystem classifies an input as “other intents,” the input is excluded from further processing. Otherwise, the input is directed to the intent decomposition and deployment subsystems. On the other hand, the UI displays the execution outcomes of the IBSM system, including intent classification results, decomposition policies, and deployment statuses.

Figure 1 illustrates the overall architecture of the IBSM system. The key technologies are discussed in detail below.

B. DUAL-PHASE JOINT INTENT CLASSIFICATION

In multi-cloud environments, applying existing methods to accurately identify diverse intents poses considerable challenges due to two main aspects. One aspect is that user characteristics substantially influence intent expression, as users vary in knowledge levels, individual expression habits, and sentimental contexts [70]. Most current research focuses on professional users who articulate intents with specialized terminology, often neglecting the needs of laypeople. Another aspect is the limitations inherent in transformer architectures, which further compound this issue. Multi-cloud environments encompass numerous basic tasks and complex task combinations. A single-phase intent classification approach

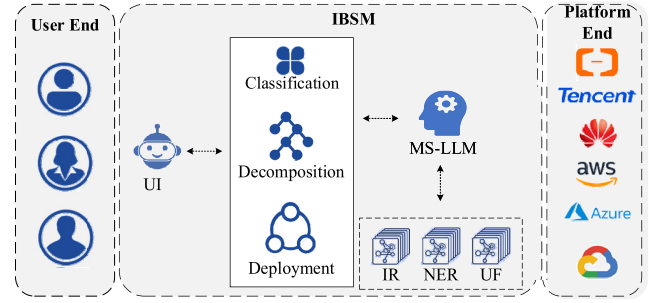


FIGURE 1. Overview of the IBSM system. The system covers the entire lifecycle of managing user intents. A user intent originates at the user end and is transmitted to the system via the user interface. Subsequently, the intent undergoes processing through intent classification, decomposition, and deployment subsystems in sequence. The first two subsystems are powered by the MS-LLM model, which fine-tunes ChatGLM4 using datasets such as IR, NER, and UF. The intent concludes at the platform end, which integrates services from multiple leading domestic and international cloud providers.

may result in excessively lengthy prompts, often surpassing the maximum input length required for optimal training.

To address these challenges, we define a characteristic parameter to identify the factors that influence intent expression. Subsequently, we introduce a dual-phase intent classification algorithm based on the MS-LLM. The algorithm decouples the storage modes and basic tasks, thereby simplifying the prompts in each phase.

Due to substantial individual variability, accurately predicting expression patterns and emotional contexts in real-world situations remains a challenging task and is beyond the scope of this paper. Consequently, the characteristic parameter (C) is solely associated with the career backgrounds (B) of individuals. Furthermore, b_T indicates a professional user, whereas b_F represents a non-expert. Therefore, C is denoted as

$$C = B = \{b_T, b_F\}. \quad (1)$$

We assume that the user provides an intent in natural language, represented by I_b , where b is an element of C . On our multi-cloud platform, each vendor supports three storage modes: block storage, object storage, and file storage. The three modes are represented by m_B , m_O , and m_F , respectively. Each mode includes a set of basic tasks. We define T as a set of basic tasks related to storage resources. Symbol T_B represents the set of basic tasks for block storage mode, T_O denotes the set for object storage mode, and T_F stands for the set for file storage mode. As a result, the set can be further refined as

$$T = T_B \cup T_O \cup T_F, \quad (2)$$

Thus, I_b is depicted as

$$I_b \in T_B \cup T_O \cup T_F. \quad (3)$$

Moreover, I_b may correspond to either a single-task set, T_S , or a multiple-task set, T_M , across different storage modes.

Thus, I_b can be further expressed as

$$I_b \in (T_{BS} \vee T_{BM}) \cup (T_{OS} \vee T_{OM}) \cup (T_{FS} \vee T_{FM}), \quad (4)$$

where T_{BS} corresponds to a single basic task for block storage and T_{BM} refers to two or more basic tasks for the same mode. Similarly, T_{OS} and T_{FS} correspond to the single basic tasks for object storage and file storage, respectively. The symbols T_{OM} and T_{FM} are related to multiple basic tasks for the modes.

According to (4), the intent recognition (IR) dataset consists of the following subdatasets: a single-intent subdataset from non-expert users (NESI), multi-intent subdataset from non-expert users (NEMI), single-intent subdataset from expert users (ESI), and multi-intent subdataset from expert users (EMI). In addition, the dataset must operate in distinct storage modes.

The proposed algorithm using the MS-LLM is outlined as follows. The algorithm takes the input I_b and outputs the corresponding basic task set T_b . In step 1, the first stage occurs. Intent I_b is mapped to one or more storage modes. This mapping is denoted as $O_1(I_b) \in \{m_B, m_O, m_F\}$. Steps 2–10 involve a secondary stage. At this stage, I_b is mapped to either T_{BS} or T_{BM} within the mode m_B . Simultaneously, if $O_1(I_b)$ contains m_O , I_b is assigned to either T_{OS} or T_{OM} . Similarly, if m_F is included, I_b is correlated with either T_{FS} or T_{FM} . Steps 11–16 check the correlation between the classification results and the current dual-phase intents of the IBSM system. If no correlation is found, the classification subsystem updates the intents before moving to the decomposition subsystem. If correlation exists, the subsystem proceeds directly to the decomposition subsystem.

Algorithm 1 Dual-Phase Joint Intent Classification

Require: MS-LLM

Input: An intent in natural language I_b

Output: Basic task set $T_b \subseteq T$

```

1: Use the MS-LLM to identify the first-level intent  $O_1(I_b)$ 
2: if the block storage mode  $m_B$  in  $O_1(I_b)$  then
3:   Use the MS-LLM to identify  $O_B(I_b)$ 
4: endif
5: if the object storage mode  $m_O$  in  $O_1(I_b)$  then
6:   Use the MS-LLM to identify  $O_O(I_b)$ 
7: endif
8: if the file storage mode  $m_F$  in  $O_1(I_b)$  then
9:   Use the MS-LLM to identify  $O_F(I_b)$ 
10: endif
11:  $O_2(I_b) = O_B(I_b) + O_O(I_b) + O_F(I_b)$ 
12: Check the correlation
13: if the correlation does not exist then
14:   Update the current dual-phase intents
15: end if
16: Move to the decomposition subsystem

```

C. COLLABORATIVE INTENT DECOMPOSITION

Mapping the original user intents to complex storage operations across heterogeneous multi-cloud platforms presents significant difficulties stemming from two factors. The first

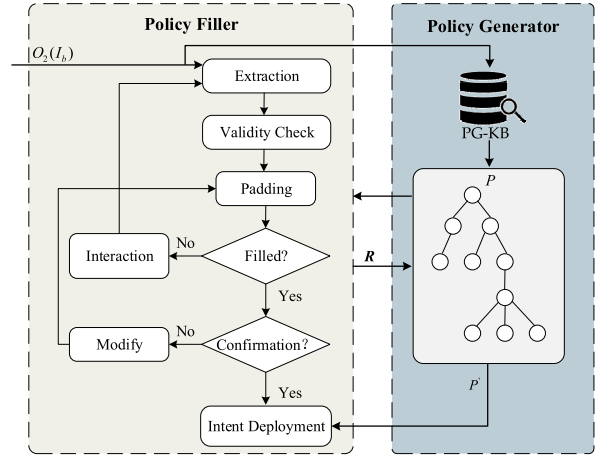


FIGURE 2. Overview of the decomposition subsystem. This subsystem consists of a policy generator and a policy filler. The generator utilizes the PG-KB, guided by $O_2(I_b)$, to construct a policy tree denoted as P . The parameters R in the tree are populated by the filler through a series of processes. The finalized policy tree, P' , is subsequently provided as input to the intent deployment subsystem.

factor is the diversity of services provided by cloud providers. For instance, Alibaba Cloud offers postpaid options for adding cloud disks, whereas Tencent Cloud supports both postpaid and prepaid options. The second is the complexity of certain basic tasks, which require numerous parameters. For example, adding a cloud disk may involve up to 40 parameters, making it difficult even for experienced users to specify all values correctly on the first attempt. As a result, user intents are often incomplete.

Although previous research has made significant contributions, most studies have focused on individual cloud providers and overlooked the issue of incomplete parameter specifications. Additionally, current methods treat each node in a policy tree as a single operation type, limiting the flexibility and control of the system.

To facilitate the thorough decomposition of user intents, a decomposition subsystem is designed to transform ambiguous user intents into a fine-grained policy tree. The subsystem collaboratively utilizes a policy generator and a policy filler. The generator constructs a fine-grained policy tree, where each node represents a basic task on a multi-cloud platform. The filler then supplements the missing parameter values for the nodes, accounting for the differences between cloud providers. This subsystem is shown in Figure 2.

1) THE POLICY GENERATOR

The generator creates a policy tree, $P = (N, E)$, based on the classification results of the user intents. In this tree, N represents the set of nodes and E denotes the directed set of edges between the nodes. Each node corresponds to a basic task in a multi-cloud platform, characterized by an atomic property. An atomic feature ensures that each basic task corresponds to a unique API within a multi-cloud platform.

More precisely, the policy trees are derived from the policy generation knowledge base (PG-KB), which is structured around the basic tasks of our multi-cloud platform. The PG-KB comprises four types of tables: intent table, task table, mapping table, and node table.

Each table is structured according to specific fields. The intent table contains information on each intent, including its identifier (ID), name, and description. The task table defines reusable basic tasks with fields, such as task ID, name, and description. The mapping table links intents to basic tasks in a defined sequence, incorporating fields such as mapping ID, intent ID, task ID, and task number. The task number reflects the order of tasks within a policy tree. The node table provides details on the API calls required for each task, including the API ID, task ID, API name, endpoint, method, and parameters.

The relationships between the tables are as follows. The intent table establishes a one-to-many relationship with the mapping table, in which each intent can be associated with multiple basic tasks. The mapping table has a many-to-one relationship with the task table, allowing multiple intents to reference the same basic task via different mappings. Lastly, the task table exhibits a one-to-one relationship with the node table because each basic task is linked to a single API. The tables and their relationships are illustrated in Figure 3.

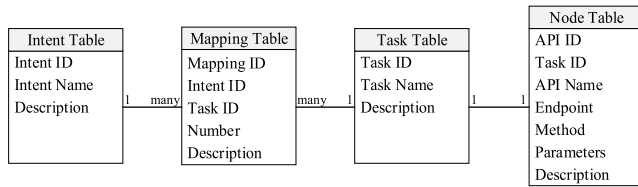


FIGURE 3. The internal structure and relationships within the PG-KB. The PG-KB encompasses four distinct table types: the intent table, task table, mapping table, and node table. The intent table establishes a one-to-many association with the mapping table, while multiple mapping tables correspond to a single task table. The task table is linked to the node table through a one-to-one relationship. Each table is systematically organized according to predefined fields.

To determine the sequence of nodes corresponding to a specific intent, the process is divided into several stages. To begin with, the policy generator queries the intent table to retrieve the intent ID by matching it to the corresponding intent name. Subsequently, the generator uses this intent ID to identify all the related basic tasks in the mapping table, sorted by the task number. Finally, the generator joins the task table with the node table in the task ID to obtain detailed task descriptions and API information in the correct sequence.

2) THE POLICY FILLER

The filler pads in the missing parameters R of the policy tree, as illustrated in Figure 3. The filler initiates the process by extracting named entity information from the user's input utilizing the MS-LLM model.

Next, the filler performs a validity check on the information using real-time data from the multi-cloud platform. It is

necessary to clarify that real-time data ensures compatibility across different cloud vendors. During the validation process, the decomposition subsystem identifies the cloud vendor, thereby guaranteeing that the named entity information is relevant to the specified vendor. Afterwards, valid information is incorporated into the policy tree.

Subsequently, the filler assesses the completeness of parameter values at each node. In the absence of any value, the filler interacts with the user to clarify their intents. This interaction is facilitated by a MC generated by the MS-LLM. Following this, the filler returns to the extraction phase to perform named entity recognition (NER) on the user's input during the interaction. Notably, prompt engineering is employed to guide the MS-LLM in generating MCs, rather than depending on specialized datasets. This method aims to reduce training costs while enhancing model controllability. Specifically, the filler generates a list of the required parameters from the policy tree, constructs a string, and integrates it into a well-designed prompt. An example of the prompt is: "You are a professional customer service representative. The current intent is $\{\}$. The list of key parameters that the user needs to provide is $\{\}$. Descriptions of parameters are enclosed in parentheses. Please inform the user that they should provide these parameters according to their descriptions." The first " $\{\}$ " is replaced with the name of the second-phase intent. The second " $\{\}$ " contains a list of parameters, including both the names of the parameters and their descriptions.

If the parameters remain partial, the filler continues to ask the user for additional input. Once all parameters are specified, the filler confirms with the user, through the MS-LLM model, whether any modifications are required. If no modifications are required, the subsystem proceeds to the intent deployment subsystem. Conversely, if adjustments are needed, the MS-LLM updates the parameters accordingly. Finally, the completed policy tree is denoted as

$$P' = (N \times R \rightarrow N', E). \quad (5)$$

D. AUTOMATED INTENT DEPLOYMENT

Striking the balance between intelligent error recovery and automated intent deployment is problematic. In the first place, multi-cloud storage operations depend on numerous APIs. A high number of APIs increases the probability of call failures. Moreover, APIs must be invoked in a specific sequence, where errors can trigger cascading effects. Finally, existing research is limited to recovery to the initial state.

To ensure the reliability of cross-cloud deployments, the intent deployment subsystem employs a closed-loop control method. This method governs the execution order of multiple APIs through the sequential processing of states within a finite-state machine (FSM). In addition, it integrates an error-handling knowledge base (EH-KB) and dedicated error-handling states, enabling checkpoint recovery. The checkpoint state represents the last state before the occurrence of an error. Figure 4 depicts the proposed method.

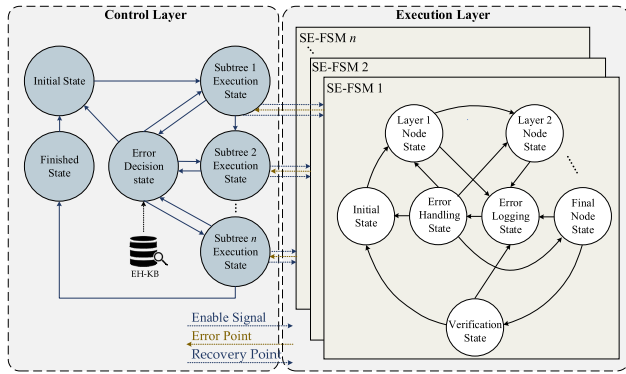


FIGURE 4. Overview of the deployment subsystem. The decomposition system consists of an LC-FSM and multiple SE-FSMs. The LC-FSM manages the execution of the entire policy tree and facilitates recovery points through the integration of the EH-KB. Meanwhile, SE-FSMs are responsible for deploying individual subtrees of the policy tree. They ensure the orderly execution of subtree nodes and enable error recovery based on the recovery points.

This method comprises of two layers: a control layer and an execution layer. The control layer is mainly composed of a loop control FSM (LC-FSM) that manages the orderly execution of different subtrees within a policy tree. This layer also monitors the entire intent deployment process. The execution layer contains several subtree execution FSMs (SE-FSMs), each responsible for controlling the sequential progression of the nodes within a subtree.

In the control layer, when the intent deployment starts, the LC-FSM enters the initial state. Once initialization is completed, LC-FSM transitions to the execution state of subtree 1 and sends an enable signal to SE-FSM 1. If SE-FSM 1 executes successfully, the LC-FSM moves on to the execution state of subtree 2. In the event of an error, the LC-FSM enters the error decision state. In this state, the LC-FSM uses error information from the SE-FSM to query the EH-KB and determine an error recovery point. EH-KB specifies the recovery points for errors that can be recovered during the execution of the policy tree. These recovery points are maintained by the development team. An introduction to the EH-KB is provided in the latter part of this section. Based on the recovery point, the LC-FSM either returns to the initial state or re-enters the execution state of subtree 1. Upon the completion of the state, the LC-FSM proceeds to the execution state of subtree 2. If this state is successfully executed, the LC-FSM transitions to the execution state of subtree 3. If an error arises, the LC-FSM re-enters the error decision state. After the execution of the subtree n is completed, the LC-FSM advances to the completion state before returning to the initial state.

The execution layer selects the appropriate SE-FSM for initialization based on the enable signal from the control layer. After initialization, SE-FSM begins by executing the first-level nodes, followed by the second-level nodes, and continues this process until it either reaches the final node or encounters an error. In the case of an error, the SE-FSM transitions to the error logging state, and subsequently to

the error handling state. During error handling, the SE-FSM refers to the recovery point provided by the control layer. From this point, the SE-FSM can either return to the initial state or resume from the checkpoint state. Once the SE-FSM safely reaches the final node, it enters the verification state. If anomalies are detected during the state, the SE-FSM moves to the error logging state. Otherwise, if no errors are found, SE-FSM returns to the initial state.

The EH-KB was developed based on the PG-KB. It is important to note that in practical applications, the EH-KB can fully encompass the functionalities of the PG-KB. Consequently, the entire IBSM system requires only one KB, which helps conserve storage space. However, for the sake of clarity, this paper presents the EH-KB separately. Based on the PG-KB, the EH-KB introduces an error point field in the mapping table to track the execution step where an error occurs. Besides, the modified structure includes a new recovery table containing several fields. The recovery ID is a unique identifier for each recovery mapping record. The error point is an integer representing the task number where an error may occur. The recovery point is an integer specifying the task number from which the process should resume if an error occurs at the error point. The table also includes the intent ID. The tables and their relationships are illustrated in Figure 5.

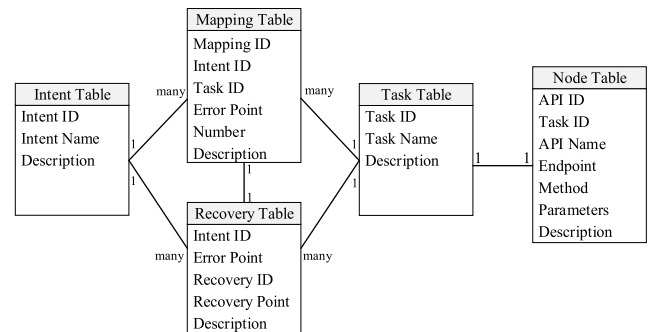


FIGURE 5. The internal structure and relationships within the EH-KB. The EH-KB builds upon the structure of the PG-KB by incorporating a recovery table. Additionally, it introduces an error point field in the mapping table. The recovery table establishes a one-to-one relationship with the mapping table.

When an error occurs, the intent deployment subsystem initiates recovery by identifying the intent ID and corresponding error point. Using these fields, the subsystem queries the recovery table to locate the recovery point associated with the detected error. This point specifies the task number that the system should resume. By resuming from the designated task, the system can proceed with the intended sequence without restarting, thereby ensuring precise and efficient error recovery.

E. THE PROPOSED MS-LLM

1) FINE-TUNING METHOD

GLM-4, developed by Zhipu AI, is the open-source iteration of the latest generation of pre-trained models in the GLM-4

series [71]. This model features only nine million trainable parameters. In evaluations across diverse datasets, including semantics, mathematics, reasoning, programming, and knowledge, GLM-4 has demonstrated strong performance. Notably, during the initial phase of this study, GLM-4-9B-Chat secured the highest position in the Compass Academic Ranking among LLMs with parameter sizes ranging from 4B to 10B [72]. Accordingly, the MS-LLM is based on GLM-4-9B-Chat. Although GLM4 is pre-trained on large-scale unlabeled data, with robust capabilities in language understanding as well as generation, additional fine-tuning is required to optimize its performance for question-and-answer tasks in the domain of cloud networking.

Fine-tuning LLMs with full parameter updates typically requires substantial computational resources, making it both costly and inefficient for many organizations. LoRA mitigates this issue by modifying the fine-tuning approach. Instead of updating all the parameters in the pre-trained model, LoRA freezes most of the model's original weights and introduces small, trainable low-rank matrices at each transformer layer. These matrices are significantly smaller than the full model weights, reducing both memory and computational demands. Furthermore, by modifying only a small subset of the model, LoRA facilitates faster convergence.

Another notable advantage of LoRA, especially in cases involving limited datasets, is its reduced susceptibility to overfitting compared to traditional full fine-tuning methods. Overfitting is a prevalent challenge when fine-tuning large models on smaller, task-specific datasets. However, by restricting the number of trainable parameters and focusing on low-rank adaptations, this issue is alleviated. As a result, models are better able to generalize, even with limited data, leading to improved performance across a range of real-world tasks.

As a result, LoRA is utilized in the fine-tuning process of MS-LLM. The specific fine-tuning parameters are outlined in Section IV. The combination of the base model and fine-tuning method presented in this paper is not intended as the sole or optimal solution, but rather as one of several potential approaches. In practice, additional modifications can be made based on the available base models and hardware limitations.

2) DATASETS

Currently, no publicly available datasets are specifically focused on multi-cloud computing. However, the methodologies used to create initial datasets in other domains provide valuable insights. For instance, Mobile-Llama adopted self-instruct method that leverages OpenAI's pre-trained models (PMs) [64]. Similarly, NetGPT adopted a self-instruct approach, leveraging OpenAI's Text-Davinci-003 model to generate relevant text samples [67]. ChatDoctor represented a pioneering effort to adapt LLMs to the biomedical domain by fine-tuning Llama using conversation demonstrations synthesized with ChatGPT [73]. DoctorGLM employed ChatGLM-6B as its base model and fine-tuned it with the

Chinese translation of the ChatDoctor dataset, generated via ChatGPT [74]. Additionally, LawyerLlama utilized ChatGPT to generate detailed explanations for each question during dataset construction [75]. In conclusion, in scenarios with limited data, domain-specific LLMs frequently rely on the ChatGPT series to create fine-tuning datasets through prompt engineering. Building on these methodologies, we develop self-constructed datasets for fine-tuning, including the IR dataset, NER dataset, and user feedback (UF) dataset.

a: IR DATASET

The IR dataset originates from basic storage tasks on our multi-cloud platform. On this platform, each vendor supports three storage modes, each encompassing a collection of basic tasks. For example, tasks associated with block storage include, but are not limited to, adding, mounting, unmounting, expanding, and deleting cloud disks.

In the process of constructing this dataset, we consider both combinations of basic tasks and the knowledge backgrounds of users. From one perspective, users often perform multiple basic tasks in combination when utilizing multi-cloud storage resources. For instance, users typically mount a cloud disk immediately after adding it. Similarly, tasks such as adding a cloud disk, creating a storage bucket, and adding a file system are often grouped together. From another perspective, the knowledge background of users significantly influences the expression of user intents. Professionals tend to use specialized terminology to articulate their intentions, whereas non-specialists usually avoid technical terms, typically using no more than one.

As a result, the IR dataset consists of the following sub-datasets: NESI, NEMI, ESI, and EMI. In addition, the dataset needs to operate in distinct storage modes. NESI aids the base model in learning to identify a single intent expressed through potentially informal or inconsistent language. NEMI plays a crucial role in training the model to recognize multiple intents within a single input, particularly when the phrasing is informal or unstructured. This dataset is invaluable for improving multi-intent classification and the model's resilience to daily expressions. ESI exposes the model to precise, well-structured expressions of single intents from expert users, thereby enhancing its understanding of technical language and improving performance in scenarios requiring accuracy. EMI supports the model in identifying multiple intents within professional language.

The process of generating the IR dataset begins by identifying the type of subdatasets. For ESI and EMI, the task name is incorporated into the base prompt, whereas for other subdatasets, the scenario name is used instead. Subsequently, the complete prompt is provided to GPT-4. Given that GPT-4 may occasionally generate errors, its outputs are manually reviewed to ensure they retain the intended meaning and complexity, while avoiding redundancy. The process concludes automatically once the specified quantity requirements for all subdatasets are met. Otherwise, generation continues until the criteria are satisfied. The process is depicted in Figure 6.

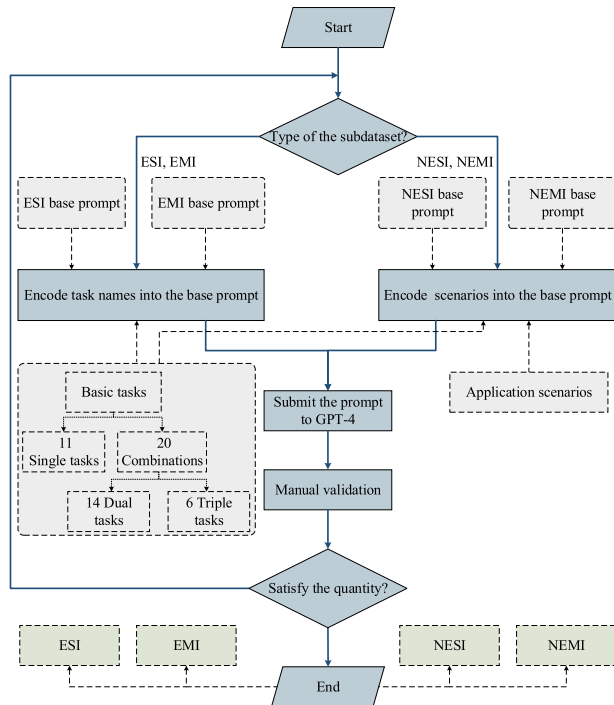


FIGURE 6. Generation of the IR dataset. The generation process begins by determining the type of subdataset, which is then used to populate the corresponding base prompt. Subsequently, the complete prompt is submitted to GPT-4. The output generated by GPT-4 undergoes manual verification. If the required quantity is achieved, the process terminates. Otherwise, it continues.

Specifically, each subdataset is associated with a distinct base prompt, which specifies either a task name or a scenario name, supplemented with text examples and corresponding requirements. Regarding task names, we identified 11 individual tasks and 20 task combinations within the multi-cloud platform. The combinations are categorized into 14 dual-task and 6 triple-task groups. The application scenarios are derived from multi-cloud platforms and the use cases of major cloud vendors. For each base prompt, we design three examples to guide text generation.

For ESI and EMI, we prioritize concise and clear language, incorporating abbreviations where appropriate, and generate 100 training samples for each basic task. The following is an example of a prompt for deleting a cloud disk: “You are a text generator. The objective is to create unique sentences where the intent is to delete a cloud disk. Examples: (1) Delete the cloud disk named mydisk. (2) Release the cloud disk named test123. (3) Remove the cloud disk named aws001. Requirements: (1) Generate 20 unique sentences with the intent to delete a cloud disk. (2) The cloud disk names should consist of alphanumeric characters, starting with a letter. (3) Ensure that each sentence maintains the intent to delete the cloud disk.”

For NESI and NEMI, the language reflects the conversational style commonly used by non-experts. Assuming the scenario involves developing a media-sharing platform, the corresponding NEMI prompt is as follows: “You are a text generator. The objective is to create unique sentences

describing a media-sharing platform. Examples: (1) On this multimedia social platform, users can easily communicate and access a wide range of audio and video content. (2) By using this creative sharing platform, users can upload and download a variety of audio-visual creations. (3) This vibrant media-sharing hub allows users to seamlessly upload and download a diverse array of audio and video content. Requirements: (1) Generate 20 unique sentences that describe the features and activities of a media-sharing platform. (2) Ensure the text reflects the context of a media-sharing platform where users upload and download content.” Each task in NESI is extended to two application scenarios, with each scenario yielding 100 training samples. Similarly, dual-task combinations in NEMI follow the same generation strategy, while triple-task combinations in NEMI are extended to five application scenarios, with each scenario producing 110 training entries. Additionally, verification and test data within each subdataset are generated proportionally, as detailed in Section IV.

b: OTHER DATASETS

The NER dataset is constructed based on API documentation maintained by developers. This documentation details the interface endpoint, request method, request examples, and request parameters associated with the basic tasks. The parameters specify the parameter name, description, type, necessity, and data type. The NER dataset extracts parameter information from user input in the form of key-value pairs, where the “key” represents the parameter name and the “value” is the parameter value. Each basic task of our platform is entered as an individual prompt into the GPT-4 model to automatically generate text as user input. For the same task, we present four input type: zero parameters, half of the parameters, the remaining half, and all the parameters. Each basic task of our platform is provided as a distinct prompt for the GPT-4 model, which autonomously generates text to simulate user input. For each task, four input configurations are employed: no parameters, partial parameters (first half), partial parameters (second half), and complete parameters.

The UF dataset captures user confirmation and modification behaviors. There are two types of user roles in this dataset. The first type includes affirmative expressions in Chinese, such as “OK” or “Continue.” The second type involves requests to modify information, such as “Change the cloud provider to Google.” Correspondingly, the assistant role has two kinds of responses. One kind is a confirmation, such as “Confirm,” whereas the other is an output in the form of a key-value pair, such as “{vendor: Google}.” The generation of user and assistant roles also depends on prompt engineering in GPT-4.

IV. EXPERIMENTS AND RESULTS

A. EXPERIMENTATION SETUP

As shown in Figure 7, the experimental setup of the IBSM system consists of a client, server, and large-scale multi-cloud

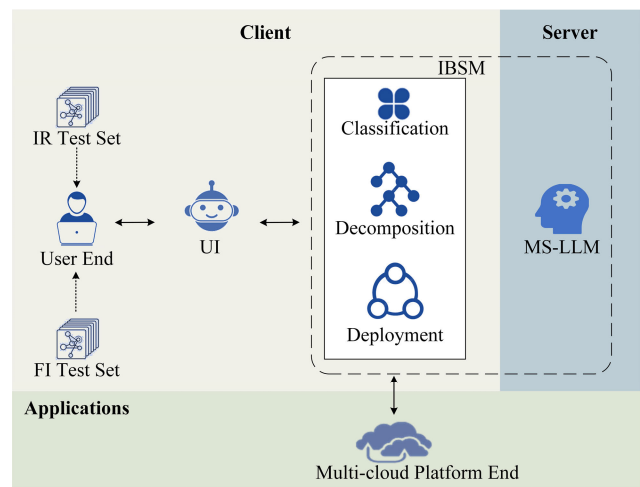


FIGURE 7. Overview of the experimentation setup. The experimental setup includes a client, a server, and a physical application. The client integrates the user interface and three subsystems. The server, equipped with an NVIDIA A100 GPU, is primarily responsible for executing the MS-LLM. The physical application represents a real multi-cloud platform.

TABLE 1. Dataset information.

Dataset	Subdataset	Training	Test	Validation
IR	ESI	1100	97	97
	EMI	2000	175	175
	NESI	2200	193	193
	NEMI	6100	535	535
NER	/	4224	700	700
UF	/	1000	800	800

exchange platform. The client, running on a 1.80 GHz Intel (R) Core (TM) i5-8250U CPU, manages user communication with the UI and operates the three main subsystems. The server, equipped with a single Nvidia A100 GPU, efficiently fine-tunes ChatGLM4 using LoRA to form the MS-LLM. Moreover, the server runs the MS-LLM to respond to client requests in real time. During the experiments, both the IR and fault injection (FI) test sets were fed into the system.

During the functional evaluation of the IBSM system, only the IR test set was employed to activate the system and sequentially trigger the three subsystems. The NER and UF test datasets were used to evaluate the MS-LLM. Detailed results of the MS-LLM evaluation are provided in a separate paper. This paper focuses on both the qualitative and quantitative assessment of the overall functionality of the system. The construction of the relevant datasets is outlined in Section III. The datasets consist of 16,624 entries, including 2,500 entries for validation, 2,500 for testing, and the remaining entries for training. Table 1 presents the distribution of entries across the training, validation, and test sets for each dataset.

The generation of the FI test set is closely linked to the LC-FSM and SE-FSMs, encompassing recoverable error types during intent deployment. These types include invalid inputs and failed API calls. The first type generally involves conflicts between user inputs and real-time data from the current cloud provider, often resulting in the failure of leaf

TABLE 2. Fine-tuning parameters.

Name	Value
max length	512
max steps	5000
learning rate	5e-5
batch size	4
LoRA r	8
LoRA alpha	32
LoRA dropout	0.1
temperature	0.95
top-p	0.95
top-k	50
max tokens	512
repetition penalty	1



FIGURE 8. Top-level view of the large-scale multi-cloud exchange platform. This platform, the first of its kind in China, exhibits strong global competitiveness. It integrates multiple cloud vendors while maintaining full neutrality. Furthermore, it provides functionalities for monitoring, statistical analysis, and evaluation of cloud network resources.

node deployment within a subtree. The second type refers to the failure to invoke the API at the final node of the subtree. The specifics of this test set, as used in the experiment, will be detailed later in this section.

Table 2 lists the fine-tuning parameters in detail. It is worth mentioning that these parameters are flexible and can be adjusted to accommodate future requirements.

Our large-scale multi-cloud exchange platform outlined in Section III is the first heterogeneous multi-cloud exchange and interconnection platform in China [76]. Compared to international companies such as Alkira and Avitrix in the emerging multi-cloud interconnection industry, its performance and number of managed cloud vendors are equivalent. The platform establishes a backbone-level network that connects multiple cloud vendors, including Alibaba, Tencent, Huawei, AWS, Microsoft Azure, and Google. Simultaneously, the platform displays statistics on current resources during operation. Additionally, it incorporates functionalities such as cost analysis, geographical distribution, and warning statistics. Figure 8 presents a top-level view of the platform.

B. EXPERIMENTATION RESULTS

1) CLASSIFICATION RESULTS

The test sets for intent classification include NESI, NEMI, ESI, EMI, and an additional category labeled “other intents.”

TABLE 3. Results of the intent classification experiments.

Model	Dataset	Accuracy	Precision	Recall
MS-LLM	NEMI	96.00%	100%	91.20%
	NESI	97.00%	98.55%	97.00%
	EMI	99%	99.70%	98.80%
	ESI	99.00%	100%	99.18%
	other	100.00%	100%	100%
GLM4-9B-Chat	NEMI	2.00%	32.5%	4.6%
	NESI	60.00%	71.82%	52.00%
	EMI	60.00%	99.5%	59.9%
	ESI	84.00%	94.36%	83.64%
	other	99.00%	100%	100%
Gemma2-9B-it	NEMI	3.00%	30.00%	7.7%
	NESI	50.00%	66.73%	40.45%
	EMI	32.00%	80.00%	32.50%
	ESI	61.00%	93.09%	60.36%
	other	100.00%	100%	100%
Qwen2-7B-Instruct	NEMI	3.00%	17.50%	6.3%
	NESI	66.00%	75.00%	61.73%
	EMI	58.00%	90.00%	57.00%
	ESI	85.00%	95.18%	84.91%
	other	100.00%	100%	100%
GPT-4	NEMI	16%	40.00%	40%
	NESI	82%	81.82%	81.82%
	EMI	100%	100%	100%
	ESI	100%	100%	100%
	other	100%	100%	100%

This category evaluates the basic understanding ability of the IBSM system, such as recognizing requests like asking for the latest movie recommendations.

We evaluate the performance of the MS-LLM against several baseline models, including GLM4-9B-Chat, Gemma2-9B-it, Qwen2-7B-Instruct, and GPT-4 [77], [78]. Among these, GLM4 serves as the base model, while Gemma2 and Qwen2 are prominent alternatives. GPT-4 represents the most advanced proprietary general-purpose model. The evaluation process is conducted for each of the LLMs.

Each LLM processes samples from distinct test sets sequentially. The classification outputs are saved in separate files for each set, which document user intent, predicted classifications, and recognition outcomes. Using these files, metrics such as accuracy, precision, and recall are computed for each test set [79]. The evaluation results are presented in Table 3.

Based on the accuracy data, MS-LLM shows a strong capability across various datasets. For the NEMI dataset, MS-LLM obtains an accuracy of up to 96%, whereas GPT-4, the best baseline model, reaches only 16%. The other baseline models score below 5%. On the NESI dataset, MS-LLM achieves 97% accuracy, compared to GPT-4's 82%, with other models below 70%. For the EMI dataset, MS-LLM realizes 99% accuracy, GPT-4 attains 100%, and other baseline models range between 60% and 32%. On the ESI dataset, MS-LLM and GPT-4 maintain accuracies of 99% and 100%, respectively, with the other models scoring between 85% and 61%. Across the other intent datasets, the models demonstrate accuracies of over 99%. In summary, the MS-LLM exhibits outstanding performance in identifying

diverse user intents across a multi-cloud environment, particularly for non-experts and in scenarios involving multiple intents.

The analysis of precision across different models shows that MS-LLM consistently achieves over 95% precision on the datasets. In contrast, GLM4 has a relatively low precision rate of 32.5% on the NEMI dataset but performs well on the EMI dataset with a precision of 99.5%. Gemma2 reaches 93.09% precision on the ESI dataset, but its performance drops to 30% on the NEMI dataset. Qwen2 maintains over 90% precision on EMI, ESI, and other intents, but only attains 17.5% precision on the NEMI dataset. GPT-4 exhibits over 100% precision on the EMI, ESI, and other intents, but only obtains 40% precision on the NEMI dataset. Therefore, MS-LLM consistently demonstrates high precision across all the datasets.

In terms of recall, MS-LLM displays exceptional performance, achieving a recall rate of over 91% across all datasets. GLM4 reveals recall rates of 52%, 59.9%, 83.64%, and 100% for NESI, EMI, ESI, and other intents, respectively. However, it performs poorly on the NEMI dataset. Gemma2 reaches a recall rate of 60.36% on the ESI dataset, but only 7.7% for NEMI dataset. Qwen2 shows a recall rate of 84.91% for ESI and 6.3% for NEMI. GPT-4 achieves recall rates of 40%, 81.82%, 100%, 100%, and 100% across the datasets. This suggests that both MS-LLM and GPT-4 display consistency and stability across the datasets.

2) DECOMPOSITION RESULTS

The policy generation primarily evaluates the functionality of the policy generator. It assesses whether the generator can

TABLE 4. The policy tree for adding and mounting a disk.

Subtree	Task Num.	Content
1	0	vendor
	1	region
	2	zone
	3	category
	4	quota
	5	charge
	6	name
	7	encryption
	8	snapshot
	9	adding disk
2	10	disk ID
	11	instance ID
	12	mounting disk

convert the user intents into a policy tree that contains a collection of basic tasks. For example, if multiple intents are to add a cloud disk and mount the disk, the generator retrieves relevant information from the PG-KB. It then produces a policy tree, which is presented in Table 4. For maintain brevity, only the basic task level is shown in the table, excluding the API level. In this example, the generator divides a tree into two subtrees. Subtree 1 focuses on adding a cloud disk and contains ten nodes. Subtree 2 is assigned to disk mounting and consists of three nodes. The results suggest that the policy generator can construct an atomic-level policy tree based on the identified intents, which is an important prerequisite for intent deployment.

The indicators for the filling process mainly assess the required time, as shown in Table 5. Intent 1 focuses on the addition of a cloud disk. Intent 2 includes both adding and mounting a cloud disk. Intent 3 involves the addition of a disk, bucket, and FS. The total numbers of policy tree nodes for these three intents is 13, 17, and 22, respectively. The input parameter “none” indicates that the professional user has not provided any parameters, whereas “full” denotes that all parameters are provided simultaneously. The term “half” indicates that only a subset of the parameters is provided. The E2E completion time of the intent flows begins at the start of intent classification and ends when the LC-FSM enters the finished state. IR time refers to the total duration required for intent classification inference. Interaction time denotes the total period spanning from the moment a question is posed to the receipt of valid information from the user. MC time represents the cumulative inference time for multi-turn conversations during the E2E process. Extraction time corresponds to the total duration needed for the policy filler to extract named entities from user inputs. This duration comprises two components: the inference time required by the MS-LLM model to execute NER tasks, and the communication latency incurred during server requests. The NER time accounts for the first component. Verification time indicates the total duration spent by the policy filler validating the extracted information. Confirmation time is defined as the

time taken by the filler to verify the details with the user. For each intent, the E2E time was tested five times, and the average time from these trials is presented in Table 5.

The interaction rate of various input parameters with the same intent shows that the average interaction time constitutes 50.14% of the E2E time. This time is positively correlated with the number of parameters that need to be added to the policy tree. When the parameters are entire, the interaction time is minimized. In cases with similar input parameters across different intents, a greater number of nodes in the policy trees is associated with longer interaction time.

The analysis reveals that, for the same intent, the MC time, extraction time and NER time are the shortest when all parameters in the policy tree are filled. Conversely, when all the parameters are pending, the times are the longest. Furthermore, the times increase as the number of nodes in policy trees grows.

An analysis of the time required for IR, verification, and confirmation within a single intent shows that the time for both processes remains unaffected by the number of parameters added. This stability occurs because, once the policy tree is generated, the parameters requiring validity checks and confirmation remain fixed. However, in experiments with varying intents, the results show that as the number of nodes in the policy tree increases, the required time also increases.

The platform logs indicate that developers spend an average of 5 minutes on intents 1 and 2. For intent 3, the average time consumed is 9 minutes. In the IBSM system, intent 1 requires a maximum of 104.24 seconds, intent 2 takes up to 133.79 seconds, and intent 3 requires no more than 219.07 seconds.

Consequently, the IBSM system can effectively execute functions described by the filler. The E2E completion time is at least half of the duration of the manual processes. Additionally, the system features a verification time in the millisecond range, along with extraction time and confirmation time in the second range.

3) DEPLOYMENT RESULTS

The analysis of the control layer shows that the execution time of the subtrees is the main factor influencing its runtime. In these experiments, the execution time of the execution layer was measured for intent 1, 2, and 3. The results were 22.23 seconds, 26.13 seconds, and 41.38 seconds, respectively. These results indicate that more nodes in a policy tree leads to a longer execution duration. The execution time of the execution layers for different intents is displayed in Figure 9.

Intent 1 includes only one subtree corresponding to nodes 0-9. Intents 2 and 3 are also included in this subtree. In the subtree, the time taken to obtain the region information relatively long, followed by the time required to get a category. This result is consistent with the data presented in Table 5. In Intent 2, nodes 10-12 form another subtree used for mounting the disk. The execution time to obtain the target disk ID is the longest, at 5.04 seconds. This delay arises because the system needs to wait for the transition of the disk’s status

TABLE 5. Results of the policy filler.

Intent	Tot. Node Num.	Input Param.	E2E (s)	IR (s)	Interaction (s)	Interaction Ratio	MC (s)	Extraction (s)	NER (s)	Verification (s)	Confirmation (s)
1	13	none	104.24	0.263	61.97	67.18%	6.442	5.58	2.55	0.16	1.27
		full	35.32		7.98	34.22%	1.207	2.93	1.12		
		half 1	78.99		38.90	58.07%	2.639	3.93	1.72		
		half 2	60.91		27.92	57.08%	5.237	3.20	1.72		
2	17	none	132.79	0.376	73.36	58.79%	7.391	10.51	4.88	0.16	2.45
		full	51.33		15.13	34.92%	1.264	4.63	2.02		
		half 1	82.16		34.55	46.59%	4.515	6.86	3.51		
		half 2	70.36		37.77	60.57%	3.523	4.21	3.28		
3	22	none	219.07	0.495	87.66	42.96%	11.737	11.14	8.87	0.16	3.21
		full	107.36		57.02	61.74%	3.936	5.09	2.83		
		half 1	184.4		71.55	42.24%	6.115	6.50	5.32		
		half 2	172.68		58.82	37.30%	8.416	7.22	5.18		

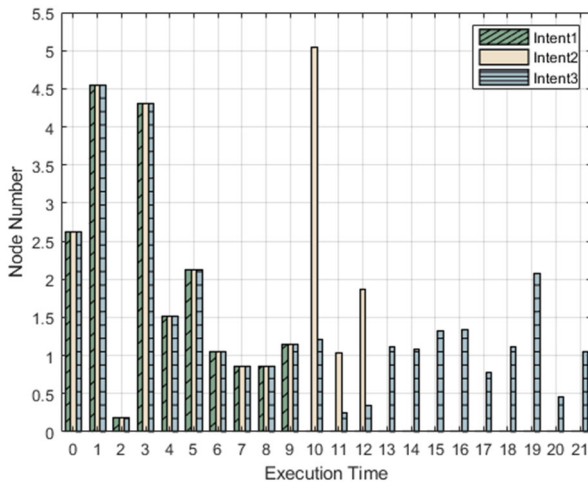


FIGURE 9. Results of the execution layer. Intent 1 involves a single task, adding a cloud disk, corresponding to nodes 0-9. Intent 2 includes two basic tasks: adding and mounting a cloud disk. The latter task is associated with nodes 10-12. Intent 3 encompasses adding a cloud disk, a storage bucket, and a file system. The latter two are linked to nodes 10-14 and 15-21, respectively. It is evident that the execution layer processes the nodes of each subtree in a sequential manner.

to “available.” In intent 3, nodes 10-14 correspond to the subtree for adding a bucket, whereas nodes 15-21 are related to the addition of an FS. It is evident that the closed-loop control method ensures orderly execution of subtrees within a policy tree.

The error recovery experiment used intent 3, which has a larger number of policy tree nodes, as an example. The experiment evaluated the E2E completion time for handling errors using checkpoint recovery (CR) and non-checkpoint recovery (NCR). In the experiment, the user input parameters are set to “none.” The results are presented in Table 6. For the subtrees corresponding to node numbers 0-9, the benefits of checkpoint recovery are not apparent. When an error occurs during the execution of the subtree related to node numbers 10-14, checkpoint recovery begins to demonstrate its advantages. In the subtree associated with nodes 15-21,

TABLE 6. Results of recovery time.

Node Num.	Error Type	CR E2E (s)	NCR E2E (s)	Decrease Ratio
0-9	invalid vendor	227.42	227.42	1.24%
	invalid region	233.05	233.05	4.41%
	invalid category	285.97	285.97	21.19%
	Failed API disk	292.13	292.13	24.64%
10-14	invalid name	313.72	240.60	23.31%
	failed API bucket	326.48	219.39	32.80%
15-21	invalid name	360.86	234.28	35.08%
	missing switch	326.81	219.39	32.87%
	failed API FS add	369.36	219.63	40.54%

the time with checkpoint recovery is considerably shorter than that without it. The reductions in the recovery time for the different error types are 35.08%, 32.87%, and 40.54%, respectively.

V. DISCUSSION

Our experimental results illustrate that the IBSM system exhibits strong intent classification capabilities across NEMI, NESI, EMI, ESI and other intents. Compared with the baseline models, the system shows greater competitiveness in NEMI. In datasets related to EMI, ESI, and other intents, the performance of the MS-LLM is on par with that of GPT-4. The performance disparity between the MS-LLM and GPT-4, particularly on the NEMI validation set, is attributed to two factors. Firstly, the NEMI dataset involves multi-intent inputs from non-experts, which involve ambiguous or loosely structured phrasing. These characteristics increases the complexity of intent classification. Secondly, the fine-tuned MS-LLM is specifically optimized for this type of data, thereby achieving superior performance in domain-specific tasks. In contrast, the pretraining and prompting strategies of GPT-4 may not be as effectively aligned with the unique linguistic features of the NEMI dataset.

Experimental results further demonstrate that the MS-LLM can refine ambiguous user intents into a more granular policy tree. The average interaction time accounts for 50.14% of the

E2E time. As additional values are required, this proportion increases. The system achieves IR and verification within milliseconds, whereas MCs, extraction, NER, and confirmation processes are completed within a few seconds. In a given policy tree, the durations for confirmation and verification are consistent. Furthermore, the system supports checkpoint recovery, resulting in an average reduction of 30.6% in error recovery time. The benefits of checkpoint recovery become more significant when errors arise toward the end of an E2E intent flow.

References [43], [44], and [45] provide important insights for our work, focusing on network management. The authors propose a policy-based approach and decompose the intents into a hierarchy of policies. They also implement closed-loop control automation. In the references, a general-purpose LLM is used with prompts to classify intents and generate policy trees. Fine-tuning LLMs are mentioned as a part of their future work. Based on these studies, we conducted research on storage resource management (SRM) in a multi-cloud environment.

However, our work differs from previous studies in several key aspects. First, we fine-tune ChatGLM4. Second, we focus more on non-experts, considering the user base of our platform. In real-world scenarios, users include both b_T and b_F . Often, b_F may have difficulty expressing their needs clearly. For example, they might express their needs in Chinese as follows: “I want to build a multimedia processing and distribution platform. Please help me create storage resources.” This means that even individuals without detailed knowledge of SRM are able to achieve their objectives merely by stating their desired outcomes. Third, we integrate error handling states into the FSMs to facilitate recovery at critical checkpoints.

Lastly, in the intent decomposition section, we identify three main distinctions. First of all, basic tasks and their combinations can be fully enumerated on our multi-cloud platform. Consequently, we do not use an LLM to generate policy trees and to repair faults, to reduce training costs and enhance response accuracy. Furthermore, we acknowledge that users may not always be able to provide all parameter values for a policy tree in advance. Even experienced users may find it challenging to manage many parameters. In addition, we directly translate user intents into atomic tasks, bypassing the usual three-step process of transforming declarative policies into definitive policies and then imperative policies.

Although our study displays promising results, several directions for future research are required. The first is the integration with broader intent-based systems. The IBSM system could serve as a foundational element in the design of a comprehensive intelligent multi-cloud resource management (IMCRM) agent. The agent would incorporate the IBSM system alongside intent-based computing management (IBCM) and intent-based network management (IBNM) systems. This integration has the potential to enable unified management of storage, computation, and network resources across heterogeneous cloud environments.

The IMCRM agent demonstrates significant potential for large-scale cloud network applications that necessitate the coordination of diverse resources. A prominent application is the EWCRT project. By integrating intent-driven resource management, the agent facilitates efficient task allocation and execution, ensuring optimal resource utilization in the western region while preventing overload in the eastern region. Furthermore, in edge computing environments, the agent can effectively balance resources between cloud and edge locations. In smart city infrastructures, which demand highly dynamic and scalable resource management, the agent supports rapid adaptation to fluctuating demand, real-time resource allocation, and recovery from system failures.

Several challenges must be addressed in deploying the IMCRM agent. As the range of applications expands, the complexity and dynamics of the environments encountered by the system will increase significantly, necessitating more efficient decision-making and enhanced adaptability. The current system relies on a static knowledge base for policy generation and error recovery, which ensures accuracy and real-time responsiveness within the existing multi-cloud platform. However, as environmental complexity grows, the knowledge base will expand exponentially, leading to a reduction in matching efficiency. In addition, large-scale distributed cloud networks are characterized by complex fault propagation chains, evolving fault modes, and high fault costs, which pose substantial challenges for existing fault management approaches.

The identified challenges have opened other directions for future research. Regarding policy generation, a comprehensive dataset will be developed, and an intelligent framework for policy creation will be established, utilizing a fine-tuned LLM. To address potential policy drift, a detection mechanism will be implemented to ensure alignment with user intents. In the area of fault management, integrating multi-dimensional fault data from intent deployment will facilitate the development of a fault prediction model based on a knowledge graph. This model will enable the IMCRM agent to proactively identify performance bottlenecks, predict issues, and trigger optimization actions, thereby mitigating the risk of failures. Additionally, an error decision dataset will be constructed to fine-tune the LLM, enhancing the agent’s ability to autonomously select the most effective recovery strategy during failures, thereby improving its self-healing capabilities.

VI. CONCLUSION

This paper proposed an IBSM system that consists of three main subsystems. These are intent classification, intent decomposition, and intent deployment. Firstly, the system builds IR, NER, and UF datasets and employs a parameter-efficient fine-tuning method based on LoRA to process the ChatGLM4. The fine-tuning process forms the MS-LLM. Secondly, the intent classification subsystem constructs the characteristic parameters of the users. The subsystem also exploits a dual-phase joint intent classification algorithm

based on the MS-LLM. The algorithm directly identifies user intents and maps them to storage modes, which are then translated into basic tasks. This remarkably improves the system's ability to classify the intents. Thirdly, the decomposition subsystem consists of a policy generator and a policy filler. The generator refines the policy tree to the atomic level. Meanwhile, the filler assists users in supplying missing parameter values in a policy tree. This subsystem ensures the production of precise low-level configurations. Finally, the deployment subsystem uses an LC-FSM along with several SE-FSMs to manage the execution order of various subtrees within the policy tree. This subsystem supports checkpoint recovery, which substantially enhances the reliability of the intent deployment. The experimental results indicate that the system facilitates a seamless intent flow from the user end to the multi-cloud platform end. The system achieves an intent classification accuracy exceeding 95%, along with a precision above 95% and a recall rate greater than 91%. In addition, the E2E time is reduced by at least 50% compared with the manual methods. The E2E time comprises 50.14% of the average interaction time. The system performs classification and verification in milliseconds, while tasks such as MCs, extraction, NER, and confirmation are executed within a few seconds. On average, checkpoint recovery time has been reduced by 30.06%. Therefore, the IBSM system is essential for enhancing autonomy in multi-cloud resource management. It has significant potential to expand the scope of multi-cloud interconnections. Future research will focus on integrating the IBSM system with IBCM and IBNM to develop the IMRMA agent, optimizing it for large-scale cloud network initiatives, such as the EWCRT project.

REFERENCES

- [1] X. Xie, Y. Han, and H. Tan, "Greening China's digital economy: Exploring the contribution of the East-West computing resources transmission project to CO₂ reduction," *Humanities Social Sci. Commun.*, vol. 11, no. 1, pp. 1–15, Mar. 2024.
- [2] National Development and Reform Commission. *The East-West Computing Resources Transmission Project: A Key Initiative for China's Digital Economy*. Accessed: Nov. 8, 2024. [Online]. Available: https://www.ndrc.gov.cn/xwdt/ztl/dsxs/zjld/202203/20220301_1317937.html
- [3] O. Tomarchio, D. Calcaterra, and G. D. Modica, "Cloud resource orchestration in the multi-cloud landscape: A systematic review of existing frameworks," *J. Cloud Comput.*, vol. 9, no. 1, p. 49, Sep. 2020.
- [4] J. Alonso, L. Orue-Echevarria, V. Casola, A. I. Torre, M. Huarte, E. Osaba, and J. L. Lobo, "Understanding the challenges and novel architectural models of multi-cloud native applications—A systematic literature review," *J. Cloud Comput.*, vol. 12, no. 1, p. 6, Jan. 2023.
- [5] S. Zhai, X. Wang, and Z. Zhang, "Analysis of multi-cloud deployment and data storage development trend against the backdrop of computing and network convergence," *ICT Policy*, vol. 49, no. 5, pp. 54–58, May 2023.
- [6] J. Hong, T. Dreiholz, J. A. Schenkel, and J. A. Hu, "An overview of multi-cloud computing," in *Proc. WAINA*, Jan. 2019, pp. 1055–1068.
- [7] *Worldwide IDC Global Datasphere Forecast, 2022-2026: Enterprise Organizations Driving Most of the Data Growth*, IDC, Framingham, MA, USA, 2022, pp. 1–2.
- [8] D. Kalla, N. Smith, F. Samaah, and E. A., "Study and analysis of ChatGPT and its impact on different fields of study," *Int. J. Innov. Sci. Res. Technol.*, vol. 8, no. 3, p. 3, Mar. 2023.
- [9] L. Pang, C. Yang, D. Chen, Y. Song, and M. Guizani, "A survey on intent-driven networks," *IEEE Access*, vol. 8, pp. 22862–22873, 2020.
- [10] A. Leivadeas and M. Falkner, "A survey on intent-based networking," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 625–655, 1st Quart., 2023.
- [11] S. Kerboeuf et al., "Design methodology for 6G end-to-end system: Hexa-X-II perspective," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 3368–3394, 2024.
- [12] A. Clemm, L. Ciavaglia, L. Z. Granville, and E. A., "Intent-based networking-concepts and definitions," in *Proc. IETF*, Wilmington, DE, USA, Dec. 2022, pp. 10–15.
- [13] S. K. Perepu, J. P. Martins, R. Souza, and K. Dey, "Intent-based multi-agent reinforcement learning for service assurance in cellular networks," in *Proc. GLOBECOM-IEEE Global Commun. Conf.*, Rio de Janeiro, Brazil, Dec. 2022, pp. 2879–2884.
- [14] C. Muonagor, M. Bensalem, and A. Jukan, "Predictive intent maintenance with intent drift detection in next generation network," 2024, *arXiv:2404.15091*.
- [15] K. Dzeperaska, A. Tizghadam, and A. Leon-Garcia, "Intent assurance using LLMs guided by intent drift," 2024, *arXiv:2402.00715*.
- [16] S. Xu, C. Kurisumoottil Thomas, O. Hashash, N. Muralidhar, W. Saad, and N. Ramakrishnan, "Large multi-modal models (LMMs) as universal foundation models for AI-native wireless systems," 2024, *arXiv:2402.01748*.
- [17] T. Metsch, M. Viktorsson, A. Hoban, M. Vitali, R. Iyer, and E. Elmroth, "Intent-driven orchestration: Enforcing service level objectives for cloud native deployments," *Social Netw. Comput. Sci.*, vol. 4, no. 3, p. 268, Mar. 2023.
- [18] C. Chung and J. P. Jeong, "A design of IoT device configuration translator for intent-based IoT-cloud services," in *Proc. 22nd Int. Conf. Adv. Commun. Technol. (ICACT)*, Feb. 2020, pp. 52–56.
- [19] C. Wu, S. Horiuchi, K. Murase, H. Kikushima, and K. Tayama, "Intent-driven cloud resource design framework to meet cloud performance requirements and its application to a cloud-sensor system," *J. Cloud Comput.*, vol. 10, no. 1, p. 30, Jun. 2021.
- [20] A. Angi, A. Sacco, F. Esposito, G. Marchetto, and A. Clemm, "NAIL: A network management architecture for deploying intent into programmable switches," *IEEE Commun. Mag.*, vol. 62, no. 6, pp. 28–34, Jun. 2024.
- [21] J. Mcnamara, D. Camps-Mur, M. Goodarzi, H. Frank, L. Chinchilla-Romero, F. Cañellas, A. Fernández-Fernández, and S. Yan, "NLP powered intent based network management for private 5G networks," *IEEE Access*, vol. 11, pp. 36642–36657, 2023.
- [22] A. S. Jacobs, R. J. Pfitscher, R. H. Ribeiro, R. A. Ferreira, L. Z. Granville, W. Willinger, and S. Rao, "Hey, lumi! Using natural language for intent-based network management," in *Proc. USENIX ATC*, Jan. 2021, pp. 625–639. [Online]. Available: <https://www.usenix.org/conference/atc21>
- [23] A. S. Jacobs, R. J. Pfitscher, R. A. Ferreira, and L. Z. Granville, "Refining network intents for self-driving networks," in *Proc. Afternoon Workshop Self-Driving Netw.*, Aug. 2018, pp. 15–21. [Online]. Available: <https://www.seldn2018.org>
- [24] M. Bezahaf, E. Davies, C. Rotsos, and N. Race, "To all intents and purposes: Towards flexible intent expression," in *Proc. IEEE 7th Int. Conf. Netw. Softwarization (NetSoft)*, Jun. 2021, pp. 31–37.
- [25] N. Vedula, N. Lipka, P. Maneriker, and S. Parthasarathy, "Open intent extraction from natural language interactions," in *Proc. Web Conf.*, Apr. 2020, pp. 2009–2020.
- [26] C. Yang, X. Mi, Y. Ouyang, R. Dong, J. Guo, and M. Guizani, "SMART intent-driven network management," *IEEE Commun. Mag.*, vol. 61, no. 1, pp. 106–112, Jan. 2023.
- [27] H. Mahtout, M. Kiran, A. Mercian, and B. Mohammed, "Using machine learning for intent-based provisioning in high-speed science networks," in *Proc. 3rd Int. Workshop Syst. Netw. Telemetry Anal.*, San Diego, CA, USA, Jun. 2020, pp. 27–30.
- [28] C. H. Cesila, R. P. Pinto, K. S. Mayer, A. F. Escallón-Portilla, D. A. Mello, D. S. Arantes, and C. E. Rothenberg, "Chat-IBN-RASA: Building an intent translator for packet-optical networks based on RASA," in *Proc. IEEE 9th Int. Conf. Netw. Softwarization (NetSoft)*, Madrid, Spain, Jun. 2023, pp. 534–538.
- [29] P. Kokinos, A. Varvarigos, D. Konidaris, and K. Tserpes, "Intent-based allocation of cloud computing resources using Q-learning," in *Proc. Int. Symp. Algorithmic Aspects Cloud Comput.*, Warsaw, Poland, Dec. 2023, pp. 184–196.
- [30] Y. Huang, H. Du, X. Zhang, D. Niyato, J. Kang, Z. Xiong, S. Wang, and T. Huang, "Large language models for networking: Applications, enabling techniques, and challenges," *IEEE Netw.*, vol. 39, no. 1, pp. 235–242, Jan. 2025.

- [31] S. B. Manir, K. M. S. Islam, P. Madiraju, and P. Deshpande, "LLM-based text prediction and question answer models for aphasia speech," *IEEE Access*, vol. 12, pp. 114670–114680, 2024.
- [32] H. Zhou, C. Hu, Y. Yuan, Y. Cui, Y. Jin, C. Chen, H. Wu, D. Yuan, L. Jiang, D. Wu, X. Liu, C. Zhang, X. Wang, and J. Liu, "Large language model (LLM) for telecommunications: A comprehensive survey on principles, key techniques, and opportunities," 2024, *arXiv:2405.10825*.
- [33] J. Ye, X. Chen, N. Xu, C. Zu, Z. Shao, S. Liu, Y. Cui, Z. Zhou, C. Gong, Y. Shen, J. Zhou, S. Chen, T. Gui, Q. Zhang, and X. Huang, "A comprehensive capability analysis of GPT-3 and GPT-3.5 series models," 2023, *arXiv:2303.10420*.
- [34] D. M. Manias, A. Chouman, and A. Shami, "Towards intent-based network management: Large language models for intent extraction in 5G core networks," in *Proc. 20th Int. Conf. Design Reliable Commun. Netw. (DRCN)*, Montreal, QC, Canada, May 2024, pp. 1–6.
- [35] D. Brodimas, K. Trantzas, B. Agko, G. C. Tziavas, C. Tranoris, S. Denazis, and A. Birbas, "Towards intent-based network management for the 6G system adopting multimodal generative AI," in *Proc. Joint Eur. Conf. Netw. Commun. 6G Summit (EuCNC/6G Summit)*, Antwerp, Belgium, Jun. 2024, pp. 848–853.
- [36] E. Jeong, H. Kim, S. Nam, J. Yoo, and J. W. Hong, "S-witch: Switch configuration assistant with LLM and prompt engineering," in *Proc. IEEE Netw. Oper. Manage. Symp.*, May 2024, pp. 1–7.
- [37] OpenAI et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [38] Y. Xu, Y. Chen, X. Zhang, X. Lin, P. Hu, Y. Ma, S. Lu, W. Du, Z. Mao, E. Zhai, and D. Cai, "CloudEval-YAML: A practical benchmark for cloud configuration generation," 2023, *arXiv:2401.06786*.
- [39] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to synthesize correct router configurations?" in *Proc. 22nd ACM Workshop Hot Topics Netw.*, Washington, DC, USA, Nov. 2023, pp. 189–195.
- [40] C. Wang, M. Scazzariello, A. Farshin, D. Kostic, and M. Chiesa, "Making network configuration human friendly," 2023, *arXiv:2309.06342*.
- [41] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostic, and M. Chiesa, "NetConfEval: Can LLMs facilitate network configuration?" in *Proc. ACM Netw.*, Mar. 2024, vol. 2, no. CoNEXT2, pp. 1–25.
- [42] B. Ifland, E. Duani, R. Krief, M. Ohana, A. Zilberman, A. Murillo, O. Manor, O. Lavi, H. Kenji, A. Shabtai, Y. Elovici, and R. Puzis, "GeNet: A multimodal LLM-based co-pilot for network topology and configuration," 2024, *arXiv:2407.08249*.
- [43] K. Dzeparoska, J. Lin, A. Tizghadam, and A. Leon-Garcia, "LLM-based policy generation for intent-based management of applications," in *Proc. 19th Int. Conf. Netw. Service Manage. (CNSM)*, Oct. 2023, pp. 1–7.
- [44] K. Dzeparoska, A. Tizghadam, and A. Leon-Garcia, "Emergence: An intent fulfillment system," *IEEE Commun. Mag.*, vol. 62, no. 6, pp. 36–41, Jun. 2024.
- [45] J. Lin, K. Dzeparoska, A. Tizghadam, and A. Leon-Garcia, "AppleSeed: Intent-based multi-domain infrastructure management via few-shot learning," in *Proc. IEEE 9th Int. Conf. Netw. Softwarization (NetSoft)*, Madrid, Spain, Jun. 2023, pp. 539–544.
- [46] J. Wang, L. Zhang, Y. Yang, Z. Zhuang, Q. Qi, H. Sun, L. Lu, J. Feng, and J. Liao, "Network meets ChatGPT: Intent autonomous management, control and operation," *J. Commun. Inf. Netw.*, vol. 8, no. 3, pp. 239–255, Sep. 2023.
- [47] L. Yao, Y. Zhang, Z. Yan, and J. Tian, "SAI: Solving AI tasks with systematic artificial intelligence in communication network," 2023, *arXiv:2310.09049*.
- [48] S. Kwon, S. Lee, T. Kim, D. Ryu, and J. Baik, "Exploring LLM-based automated repairing of ansible script in edge-cloud infrastructures," *J. Web Eng.*, vol. 22, no. 6, pp. 889–912, Dec. 2023.
- [49] S. K. Singh, S. Kumar, and P. S. Mehra, "Chat GPT & Google bard AI: A review," in *Proc. Int. Conf. IoT, Commun. Autom. Technol. (ICICAT)*, Gorakhpur, India, Jun. 2023, pp. 1–6.
- [50] S. K. Mani, Y. Zhou, K. Hsieh, S. Segarra, T. Eberl, E. Azulai, I. Frizler, R. Chandra, and S. Kandula, "Enhancing network management using code generated by large language models," in *Proc. 22nd ACM Workshop Hot Topics Netw.*, Cambridge, MA, USA, Nov. 2023, pp. 196–204.
- [51] D. Donadel, F. Marchiori, L. Pajola, and M. Conti, "Can LLMs understand computer networks? Towards a virtual system administrator," 2024, *arXiv:2404.12689*.
- [52] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [53] D. Michael Manias, A. Chouman, and A. Shami, "Semantic routing for enhanced performance of LLM-assisted intent-based 5G core network management and orchestration," 2024, *arXiv:2404.15869*.
- [54] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, "Mistral 7B," 2023, *arXiv:2310.06825*.
- [55] M. A. Habib, P. E. I. Rivera, Y. Ozcan, M. Elsayed, M. Bavand, R. Gaigalas, and M. Erol-Kantarci, "LLM-based intent processing and network optimization using attention-based hierarchical reinforcement learning," 2024, *arXiv:2406.06059*.
- [56] N. Van Tu, J.-H. Yoo, and J. W.-K. Hong, "Towards intent-based configuration for network function virtualization using in-context learning in large language models," in *Proc. NOMS-IEEE Netw. Oper. Manage. Symp.*, May 2024, pp. 1–8.
- [57] O. G. Lira, O. M. Caicedo, and N. L. S. da Fonseca, "Large language models for zero touch network configuration management," 2024, *arXiv:2408.13298*.
- [58] L. Tunstall, E. Beeching, N. Lambert, N. Rajani, K. Rasul, Y. Belkada, S. Huang, L. von Werra, C. Fourrier, N. Habib, N. Sarrazin, O. Sansevero, A. M. Rush, and T. Wolf, "Zephyr: Direct distillation of LM alignment," 2023, *arXiv:2310.16944*.
- [59] A. Fuad, A. H. Ahmed, M. A. Riegler, and T. Čičić, "An intent-based networks framework based on large language models," in *Proc. IEEE 10th Int. Conf. Netw. Softwarization (NetSoft)*, Vienna, Austria, Jun. 2024, pp. 7–12.
- [60] S. Long, F. Tang, Y. Li, T. Tan, Z. Jin, M. Zhao, and N. Kato, "6G comprehensive intelligence: Network operations and optimization based on large language models," 2024, *arXiv:2404.18373*.
- [61] D. Wang, Y. Wang, X. Jiang, Y. Zhang, Y. Pang, and M. Zhang, "When large language models meet optical networks: Paving the way for automation," *Electronics*, vol. 13, no. 13, p. 2529, Jun. 2024.
- [62] A. Mekrache and A. Ksentini, "LLM-enabled intent-driven service configuration for next generation networks," in *Proc. IEEE 10th Int. Conf. Netw. Softwarization (NetSoft)*, Vienna, Austria, Jun. 2024, pp. 253–257.
- [63] A. Mekrache, A. Ksentini, and C. Verikoukis, "Intent-based management of next-generation networks: An LLM-centric approach," *IEEE Netw.*, vol. 38, no. 5, pp. 29–36, Sep. 2024.
- [64] K. B. Kan, H. Mun, G. Cao, and Y. Lee, "Mobile-LLaMA: Instruction fine-tuning open-source LLM for network analysis in 5G networks," *IEEE Netw.*, vol. 38, no. 5, pp. 76–83, Sep. 2024.
- [65] S. Zhang, L. Dong, X. Li, S. Zhang, X. Sun, S. Wang, J. Li, R. Hu, T. Zhang, F. Wu, and G. Wang, "Instruction tuning for large language models: A survey," 2023, *arXiv:2308.10792*.
- [66] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 661–678.
- [67] Y. Chen, R. Li, Z. Zhao, C. Peng, J. Wu, E. Hossain, and H. Zhang, "NetGPT: An AI-native network architecture for provisioning beyond personalized generative services," *IEEE Netw.*, vol. 38, no. 6, pp. 404–413, Nov. 2024.
- [68] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.
- [69] *The Future Network White Paper: Multi-Cloud Switching Networks*, Purple Mountain Laboratory, Nanjing, China, 2022, pp. 8–35.
- [70] Y. Du, T. Li, M. S. Pathan, H. K. Teklehaimanot, and Z. Yang, "An effective sarcasm detection approach based on sentimental context and individual expression habits," *Cognit. Comput.*, vol. 14, no. 1, pp. 78–90, Jan. 2022.
- [71] T. Glin et al., "ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools," 2024, *arXiv:2406.12793*.
- [72] Open Compass. *Compass Rank*. Accessed: Aug. 5, 2024. [Online]. Available: <https://opencompass.org.cn/home>
- [73] Y. Li, Z. Li, K. Zhang, R. Dan, S. Jiang, and Y. Zhang, "ChatDoctor: A medical chat model fine-tuned on a large language model meta-AI (LLaMA) using medical domain knowledge," *Cureus*, vol. 15, no. 6, p. 40895, Jun. 2023.
- [74] H. Xiong, S. Wang, Y. Zhu, Z. Zhao, Y. Liu, L. Huang, Q. Wang, and D. Shen, "DoctorGLM: Fine-tuning your Chinese doctor is not a herculean task," 2023, *arXiv:2304.01097*.
- [75] Q. Huang, M. Tao, C. Zhang, Z. An, C. Jiang, Z. Chen, Z. Wu, and Y. Feng, "Lawyer LLaMA technical report," 2023, *arXiv:2305.15062*.

[76] *Purple Mountain Maboratory Has Garnered Attention From National Media*. Accessed: Aug. 5, 2024. [Online]. Available: https://www.sohu.com/a/620511276_120991886

[77] G. Team et al., “Gemma 2: Improving open language models at a practical size,” 2024, *arXiv:2408.00118*.

[78] A. Yang et al., “Qwen2 technical report,” 2024, *arXiv:2407.10671*.

[79] J. Liu, Y. Li, and M. Lin, “Review of intent detection methods in the human-machine dialogue system,” *J. Phys., Conf. Ser.*, vol. 1267, no. 1, pp. 1–10, Jul. 2019.



SHUXIN QIN received the Ph.D. degree in computer application technology from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 2014. He is currently a Researcher with the Purple Mountain Laboratories, Nanjing, China. His research interests include deep learning and data mining.



JINGYA ZHENG received the Ph.D. degree in computer application technology from the National Space Science Center, Chinese Academy of Sciences, Beijing, China, in 2023. She is currently a Researcher with Shandong Future Network Research Institute, Jinan, China. Her current research interests include intelligent cloud-network integration, intent-based networking, and computing-first networking.



DAN WANG received the M.S. degree in information and communication engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2020. She is currently with the Purple Mountain Laboratories. Her research interests include cloud computing and natural language processing.



GAOFENG TAO received the M.S. degree from Beijing Institute of Technology, Beijing, China, in 2003. He is currently an In Charge of major research subjects with the Purple Mountain Laboratories, Nanjing, China. His research interests include deterministic networks, cloud-network integration, cloud computing, and deep learning.



ZHONGJUN MA received the M.S. degree in electronics and communication engineering from Shandong University, Jinan, China, in 2012. He is currently with Shandong Future Network Research Institute. His primary research interests include access network transmission technologies and deterministic networking.

...